Tutorial 8 Script

Data repositories and Querying

Hi everyone, In today's tutorial we're going to talk about how to query data that is stored in different kinds of data repositories.
We'll start with SQL, which is the standard way to query relational databases, and then we will be looking at how a NoSQL database like MongoDB expresses similar ideas in a different syntax. The goal here is to make sure you will have a basic understanding of the shape of the queries you'll need to perform basic data querying.

Page 1 Outline
On this slide you can see a few typical database systems: MySQL, SQL Server, Oracle, PostgreSQL, and so on.

All of these support SQL, and the core ideas are the same, but sometimes the function names or keywords differ slightly.

The slide also lists where to find installation instructions for different operating systems — that's just to show you SQL is cross-platform; you don't need to memorise the links. In practice, you always check the official docs for the exact command.

Page 2 SQL Query
On this slide we have the sample data that all the later SQL examples are based on. There are two tables: The EmployeeInfo and the EmployeePosition.

The first table mainly tells us who the employee is and which department/ project they belong to. Each row here is a person. And there's all the attribute fields of the person's information.

The second table shows what position that employee holds and how much they earn.

(Below is explanations of what is a relational model)
This is what SQL dataset looks like. At its core, SQL uses the relational model. That means that at a is organised into relations, and a relation is what we see here as a table.

A table has a schema, which is the fixed set of columns and their types, and the rows inside the table all follow this schema. Every row has the same columns, in the same order, with the same meaning.

The reason we can link EmployeeInfo and EmployeePosition is that they share the same logical identifier, EmpID.
In the relational model, this is how you express connections: you don't nest one

record inside another, you keep data in separate tables and relate them through keys.

Typically, one table will have a primary key (like EmpId in the employee table), and another table will store that key as foreign key to say “this row belongs to that employee”

So to summarise, because of this design, SQL is very good at questions like “from this table, give me all rows that match that table”, which means a joins operation is being performed during this process. And all the queries on the later slides of SQL are just reading and filtering this structured, schema-first data.

Page 3 SQL (Q1, Q2)
On this slide we start doing actual queries on the EmployeeInfo table, and these two questions are both very basic but very representative: one is about **transforming** a column and giving it an **alias**, and the other is about **counting with a condition**.

What this is testing is: you can apply a function to a column, and you can rename the output column in the result set.

As we can see in this first command, written like this,
- UPPER() is a string function that turns the first name into all capital letters.
- AS is a keyword that assigns an alias, like what we do here is just renaming the column in the result, so when you see the output, the column header will say EmpName instead of UPPER(EmpFname)
- FROM is a keyword that tells SQL which table to read from.

So the idea is: SELECT, and then apply function, give a nicer name, and finally tell SQL from which table these selections are happening.

Then we take a look at the second command. Here this question switches from show me values to tell me how many. This means we need an aggregate function, and the simplest one is COUNT(). The COUNT function counts how many rows match the condition. And WHERE is a keyword that limits it to only those rows where the department is exactly “HR”. So WHERE assigns the condition.

when the question says “how many…”, think COUNT, and when it says “how many… in a certain group,” add a WHERE. The table is the same as before — we’re just asking it a different kind of question.

Page 4 SQL (Q3)
This question is just showing that SQL has built-in date/time functions, and you can select from them directly without a table.

You should use GETDATE() function in SQL Server, and SYSTDATE() in MySQL

So it's the same thing, just different function names in different systems. The main point is: **use SELECT + a built-in function to get the current date.**

Page 5 (Q4, Q5)
On this slide we have two very similar questions, both using string functions. The idea is: sometimes the value in a column is longer than what we actually need, so we use SUBSTRING to take only part of it.

For the command here for Q4, obviously under the same SQL command skeleton, where we use SELECT and FROM, etc., we used the SUBSTRING() function to take only parts of a certain string. The three arguments is column, starting index and length of substring, respectively. Take notice that the index here does NOT start at 0.

For the command here for Q5 we're asked to fetch only the place name (the part before the brackets) from the Address column.
We can still use SUBSTRING, but this time we don't hard-code the length, instead we *find* where the left bracket is, and cut up to there.

Here we use the function CHARINDEX(), it tells us the position of the designated character. In this case it tells us the position of the left bracket in that address string, then SUBSTRING takes everything from the start up to that position.

Page 6 (Q6, Q7, Q8)
This slide is three small but very "exam-style" SQL tasks.

Q6. Write a query to create a new table which consists of only structure copied from the other table.
Here we want to make a table that looks like EmployeeInfo but has no rows. One trick is to use SELECT … INTO … but make the SELECT return zero rows:
- SELECT * INTO NewTable FROM EmployeeInfo means "create NewTable with the same columns as EmployeeInfo."
- WHERE 1 = 0 is always false, so no data is actually inserted.

So the result is: **new table, same schema, empty.** That's exactly what the question wants

Q7. Write a query to find all the employees whose salary is between 50000 and 100000.
Now we move to the EmployeePosition table because salary is stored there.
The natural way to write a range in SQL is BETWEEN … AND …:
- This means salary larger than or equals fifty thousand and salary less than or equals one hundred thousand.

In the slide they put the numbers in quotes, but if the column is numeric you

don’t need the quotes — keeping them as numbers is cleaner.

And Q8. Write a query to find the names of employees that begin with ‘S’.
In this one we use the LIKE keyword, in this case, 'S%' means: string starts with S, followed by any number of characters.
Note that LIKE is not a full regular expression engine — it only supports simple wildcards such as % for “any length” whereas real regex supports many more patterns.

Page 7 SQL (Q9, Q10)
Q9 is a “give me the highest-paid people” question.
The key idea is always the same: **sort by salary in descending order first, then take only N rows.** Different SQL dialects just write the “take N rows” part differently.
Here shows the different command for SQL Server and MySQL, so: same logic, different keyword. The important part is the ORDER BY Salary DESC; without that, “top N” doesn’t make sense.

In Q10 Write a query to retrieve the EmpFname and EmpLname in a single column as FullName, here we want to **join two columns into one string** and give it a nicer name. We can use CONCAT and add a space in between
- CONCAT() function glues the pieces together,
- the blank space is the space between the first and the last name
- the AS keyword, as introduced above, is the alias so the result column is called FullName.

Page 8 SQL (Q12, Q13, Q14)
Then let’s take a look at Q12, 13, and 14.

Q12. “Write a query to fetch all the records from the EmployeeInfo table ordered by EmpLname in descending order and Department in ascending order.”
Here we’re practicing multi-column sorting. SQL lets you sort by more than one column, and you can give each column its own direction.
In this command, first it will sort by last name from Z to A. If two people have the same last name, then it will sort those people by department from A to Z. So the order is decided left to right in the ORDER BY list.

Q13. “Write a query to fetch details of employees whose EmpLname ends with ‘a’ and contains five alphabets.”
This one combines pattern matching and fixed length.
In this command we use the LIKE keyword as above. And the pattern is written like this. Each _ in SQL LIKE means “exactly one character,” and % means “any number of characters.” So '____a' means: 4 characters, then an a → total length 5, ending in a.
(And remember: LIKE here is simple wildcard matching, not full regex.)

Q14. “Write a query to fetch details of all employees excluding the employees with first names ‘Sanjay’ and ‘Sonia’.”
This shows how to filter out a small list of values using NOT IN
So we return every row except the ones whose first name is exactly Sanjay or exactly Sonia.

Page 9 SQL (Q15, Q16, Q17)
Q15. “Write a query to fetch details of employees with the address as DELHI(DEL).”
Here we’re matching a value that has a fixed prefix (DELHI(DEL)) and possibly more after it. That’s why the slide uses LIKE with a %:
'DELHI(DEL)%' means the address starts with DELHI(DEL) and can have anything after that.
This is the same simple wildcard matching we talked about earlier: % = any length, not full regex.

Q16. “Write a query to fetch all employees who also hold the managerial position.”
Now we need information from **both** tables: the personal info is in EmployeeInfo, but the position is in EmployeePosition. So we join them on EmpID, and then filter for “Manager”.
In this command, E and P are just short names (aliases) for the two tables.
INNER JOIN ... ON E.EmpID = P.EmpID links the employee to their position record.
WHERE P.EmpPosition IN ('Manager') keeps only those rows where the position is Manager.
So the result is “people, but only those who are managers.”

Q17. “Write a query to fetch the department-wise count of employees sorted by the department’s count in ascending order.”
This is a **grouping** question. We want to count how many employees there are in each department.
Using this command, GROUP BY Department forms a group for each department.
COUNT(EmpID) counts how many employees are in that group.
ORDER BY EmpDeptCount ASC sorts from the smallest department to the largest.
This shows the typical pattern: first, **GROUP BY → then, aggregate → next, ORDER BY the aggregate.**

Page 10 (Q18)
This question is a bit different from the previous ones because it’s not asking “which employees meet a business condition,” it’s asking “how do I pick even-numbered rows or odd-numbered rows from a table.” SQL tables don’t naturally have a built-in row order, so the usual trick is: first create a row number, then filter on that number.

First the even numbers. What's happening?

1. The inner query pretends every row has a number (rowno). In some databases you'd generate this with a window function like ROW_NUMBER(), here it's just shown conceptually.
2. In the outer query we use MOD(rowno, 2) = 0 to say "keep rows whose row number is divisible by 2" → that's the even rows.

If the table does not originally have a `rowno` column, writing

sql 复制

```
SELECT rowno, EmpID
FROM EmployeeInfo;
```

will *not* magically produce a `rowno` in most SQL databases. You must use whatever "row-numbering" feature your database provides to generate that column first. The example in the slide is illustrating the idea — "first get a row number, then use MOD to separate even and odd rows" — but it's not a query that works exactly the same in every SQL dialect.

Similar idea applies to the case where you need odd row numbers.
In practice, we first add a row number using ROW_NUMBER() (or an equivalent feature in your database), and then we apply MOD on that generated number.

Page 11 SQL Practice
Now look at Q20, think about it for a while. Try write your own answers, and we'll continue shortly.

[2 mins]

This practice question says: "Write a query to retrieve the list of employees working in the same department."
What makes this different from the earlier department questions is that here we want to **compare employees to other employees** in the same table. There isn't a second table that stores "department memberships," so we have to match the table to itself — that's called a **self-join**.

The answer here is like this

Proceed to Page 11 SQL Practice
Here's what each part is doing:

- EmployeeInfo E, EmployeeInfo E1 means we are taking the same table twice and giving it two different aliases, E and E1. You can think of it as "employee A" and "employee B."
- WHERE E.Department = E1.Department keeps only the pairs where the two employees are in the **same** department.
- AND E.EmpID != E1.EmpID removes the trivial case where we matched the same person to themselves.
- We select from just E (and use DISTINCT) so that each employee in

such a department appears once in the result.

That's the standard pattern when a question says "find people who are in the same X" but all the information is in one table.

Page 12 SQL Practice
Here is another practice. Take a look.

[2 mins]

This question is: "Write a query to find the third-highest salary from the EmpPosition table."
The key idea is: to get the 3rd highest, we first have to sort salaries from **highest to lowest**, pick the **top 3**, and then from those 3 keep only the **smallest** one — because among the top 3, the smallest is exactly the 3rd highest.

Here's the logic step by step:
1. **Inner query**: This takes the salaries, orders them from largest to smallest, and keeps only the first 3. So now we have "highest, 2nd highest, 3rd highest".
2. **Outer query**: Now we take those 3 numbers and sort them **ascending** (smallest first). Among those three, the smallest one is the 3rd highest overall. Then TOP 1 picks that one.

Page lost NoSQL Query
On this slide we switch from SQL to NoSQL, and the example database is MongoDB.
Up to now we've been looking at relational tables — rows and columns. In MongoDB the basic unit is not a row but a **document**. A document is shown in JSON format, like the one on the slide

Here we have a document in the customer collection:

A few things to notice:

- It's **key–value** pairs, not fixed columns.
- There is an automatic `_id` field that uniquely identifies the document — this plays a similar role to a primary key in SQL.
- Other fields like `name`, `age`, `gender`, `amount` can be whatever your application needs.

So the main point of this slide is: in MongoDB, data is stored as JSON-like documents inside a collection, instead of as rows inside a table."

Page lost NoSQL
On the previous slide we saw what a MongoDB document looks like. Now we want to retrieve a specific document — the one where the customer's name is John.

In MongoDB we do that with the find method, and we pass a JSON object that describes the condition:

- `db.customer` means “use the `customer` collection” — this is like choosing a table in SQL.
- `.find(...)` means “search in this collection.”
- `{ name: "John" }` is the filter: only return documents whose `name` field is exactly `"John"`.

Page lost NoSQL
This slide just shows a nicer way to display the same result.
We still query the customer collection for the document where name is "John": .pretty() doesn’t change the data — it only formats the JSON output so it’s easier to read in the shell: each field on its own line, with indentation.

Page lost NoSQL
“Now we want to query customers who are **older than 40**. In MongoDB we do that by putting a comparison operator inside the query object.

Here’s what’s happening:

- We’re still querying the `customer` collection.
- The field we care about is `age`.
- Instead of writing `age: 40`, we write `age: { $gt: 40 }`.
- `$gt` stands for **greater than**.

So this reads as: ‘find documents where the age field is greater than 40.’
Then .pretty() is again just to format the output nicely.

in MongoDB, numeric comparisons are done with these dollar-prefixed operators — $gt (greater than), $lt (less than), $gte (greater or equal), $lte (less or equal) — all written inside the field’s value.

Page lost NoSQL
Here we combine two conditions: we want **female** customers, and we also want them to be **younger than 25**
In MongoDB, when you put multiple field conditions inside the same object, it’s treated as an **AND** — all of them must be true. So we can write:

- `gender: "female"` filters by gender.
- `age: { $lt: 25 }` says the age must be less than 25 (`$lt` = “less than”).
- Putting them together in one object means “female AND age < 25”.

Page lost NoSQL
This slide repeats the previous query but shows the more explicit way of writing an AND in MongoDB

Before, we relied on the fact that

```js
{ gender: "female", age: { $lt: 25 } }
```

is treated as “gender = female AND age < 25”.

Here we write it using the $and operator:

```js
db.customer.find({
  $and: [
    { gender: "female" },
    { age: { $lt: 25 } }
  ]
}).pretty()
```

- $and says “all of the conditions in this list must be true.”
- Then we put each condition as a separate object inside an array.
- The actual conditions are the same as before.

So this form is more verbose, but it’s useful when you have many conditions or when you mix AND with OR. Conceptually it’s exactly the same as the previous slide.”

Page lost NoSQL Practice
Now let’s take a look at this practice.

[2 mins]

“The task here is: query customers who are **either male OR younger than 25**. This time we are not doing an AND — we don’t require both conditions at the same time. We want documents that satisfy at least one of them. So we use MongoDB’s $or operator.

The query looks like this:

```js
db.customer.find({
  $or: [
    { gender: "male" },
    { age: { $lt: 25 } }
  ]
}).pretty()
```

- `$or` means: return a document if **any** of the conditions in the array is true.
- The first condition keeps all male customers.
- The second condition keeps anyone whose age is less than 25.
- Putting them under `$or` combines them.

So compared with the previous slide, the only change is `$and` → `$or`, and that switches the logic from “both must be true” to “at least one must be true.”